

cpre488-mp2

Report

Completed tasks:

- ✓ [detailed system diagram](#)
- ✓ [changes made to camera_app.c](#)
- ✓ [why at this point, camera had no color](#)
- ✓ We convert the 8-bit output of the “Video In to AXI4-Stream” IP core to 16-bits to be given to the VDMA by appending the 8-bit value “10000000” (see step 3.vii). Explain why this is an appropriate value to append, and why appending “00000000” would not make sense.
- ✓ Note that several of these ports in the XDC file are paired together, with one port ending in `_p` and the other ending in `_n`. In your writeup, briefly describe what this pairing of signals signifies, and what this configuration is typically used for.
- ✓ [starter hardware operation intentions](#)
- ✓ Bonus Credit: Various Analog and Digital Adjustments

Completed But need checks:

- ✓ The YCbCr 4:2:2 pattern is an example of an encoding scheme referred to as chroma subsampling: http://en.wikipedia.org/wiki/Chroma_subsampling#4:2:2. Because the human visual system is less sensitive to the position and motion of color than it is to luminance, bandwidth can be optimized by storing more luminance detail than color detail. Look at the VDMA initialization code in function `fmc_imageon_enable()`, and infer from the Red, Green, and Blue examples how the 16-bit 4:2:2 YCbCr format is encoded. Briefly describe this in your writeup, and use this format as the output of your `camera_loop()` conversion pass.

Tasks:

- ☐ The image pipeline should proceed from `vita` -> `vid_in` -> `v_demosaic_0` -> `v_proc_ss0` (Color Conversion Only) -> `v_proc_ss1` (422-444 Chroma Resampling Only) -> `axis_subset_converter` -> `vdma_S2MM` -> `vdma_MM2S` -> `vid_out` -> `hdmi_out`. Provide a diagram for this awesome pipeline in your writeup, making sure to label the bit width of the relevant signals.
- ☐ Describe performance
 - ☐ Software Pipeline
 - ☐ Performance
 - ☐ Testing Methodology

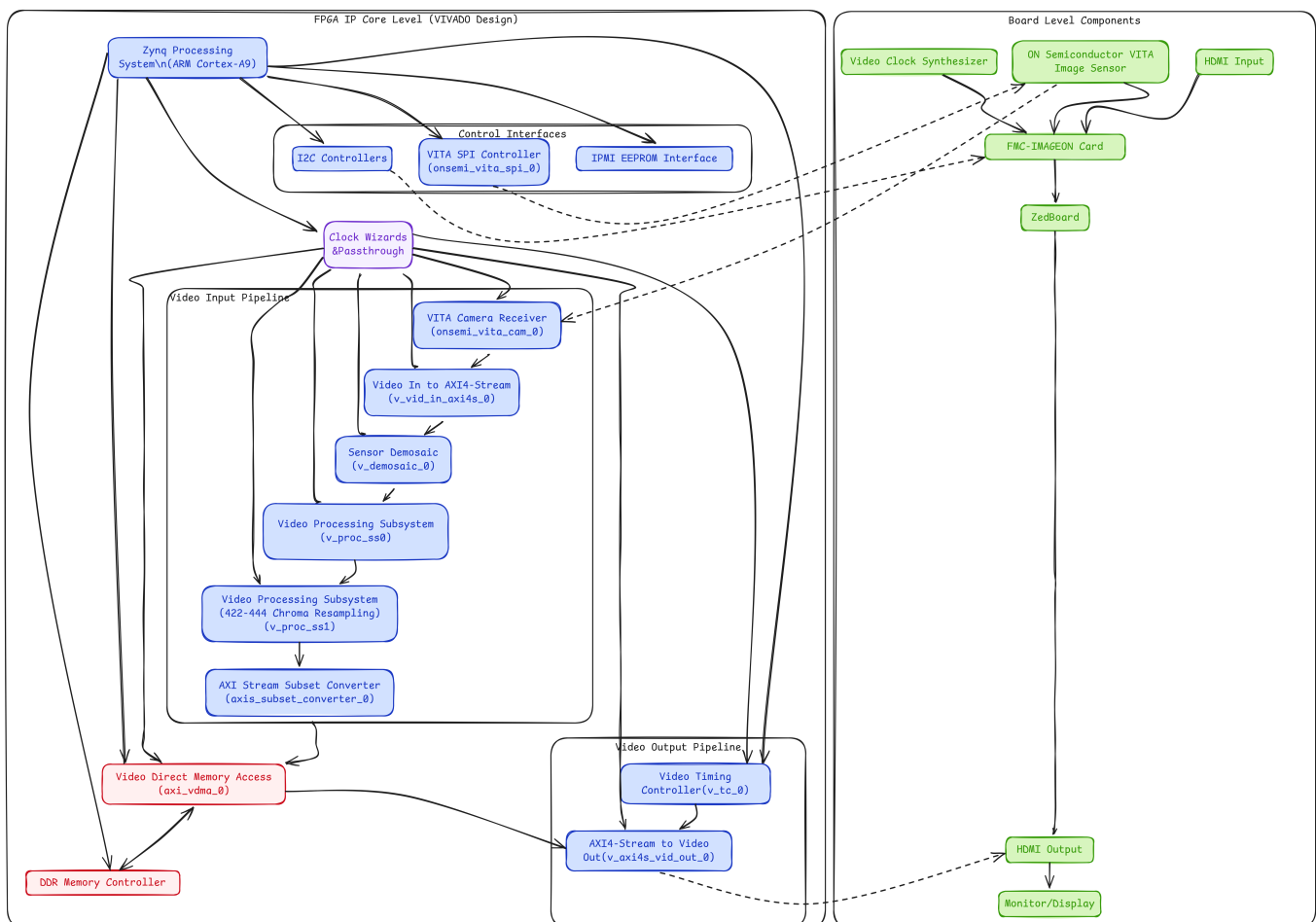
- ☐ Hardware Pipeline
- ☐ Performance
- ☐ Testing Methodology

- ☐ Bonus Credit: Video Mode
- ☐ Bonus Credit: Digital Zoom Mode

Detailed System Diagram

The following diagram illustrates the interconnection between the various modules in the system, both at the IP core level (i.e. the components in our VIVADO design) as well as the board

level (i.e. the various chips that work together to connect the output video to the monitor).



Starter Hardware Operation Intentions

The overall goal of the starter hardware is to provide an interface to FMC device such that a test image sequence can be displayed over the HDMI port on the FMC device. To accomplish this, a Test Pattern Generator IP is instantiated and configured using the AXI bus to produce a video stream that is provided to the VDMA. The VDMA is configured to store this stream to a memory location and forward the stream to an AXI Stream output IP block, which passes the

stream to the AVNET HDMI Output IP block. This gives the test pattern stream a direct path to the FMC module so it can be displayed.

However, we also need to incorporate timing information. Similar to the VGA protocol, HDMI requires timing signals to make sure line draws are all synced up. To do this a Video Timing Controller IP block is used. This IP block is configured off of an AXI bus fed to it and it outputs all the timing signals that the HDMI IP block needs. These timing signals are fed into the AXI Stream to Video Out IP block, which then forwards the timing signals to the AVNET HDMI Output.

In addition, there are two I2C IP blocks, the FMC IPMI ID EEPROM I2C block and the FMC IMAGEON I2C block. The purpose of the IMAGEON interface is to provide a way for the ZYNQ processor to control the FMC peripheral. Then, the purpose of the EEPROM I2C interface is to provide the ZYNQ processor a way to configure the on-board EEPROM on the FMC, which stores important information.

For the VDMA, the primary difference between this setup and the setup from MP-0 is that the VDMA is configured for both reads and writes. There is a stream incoming from the TPG that is written to memory, and then that memory is read out to the HDMI. This requires GenLock synchronization between the reads and writes, which was not needed in MP-0.

Finally, there are two clock domains defined for this design, a 100MHz clock and a 148Mhz clock. The 100MHz clock is used for all the AXI bus transactions and is considered the primary clock. Then the 148MHz clock is used for the video clock. Looking at the block diagram, all modules that are fed a video stream use this clock and this clock is passed directly to the AVNET HDMI IP block. So, it is safe to say that the purpose of the 148MHz clock is to clock the video streams.

This design only allows for the display of the test pattern, so we need to add more IP cores later to use the camera.

What are the changes we made to `camera_app.c` ?

TPG Change

For the TPG change, we referenced the provided datasheet to see what we configure via memory mapped registers. We saw that we had the ability to set a foreground and background to be a variety of preset patterns. So, we set the background to a colored bars pattern (register value `0x9`) and the foreground to be a colored box that bounces around (register value `0x1`). Since we were enabling a box, we had to specify its dimensions and colors, which was simple to do since there were registers for each. The relevant code for this update is shown below:

```

1 // Define convenient volatile pointers for accessing TPG registers
2 volatile uint32_t *TPG_CR      = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0); // TPG Control
3 volatile uint32_t *TPG_Act_H   = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x10); // Active Height
4 volatile uint32_t *TPG_Act_W   = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x18); // Active Width
5 volatile uint32_t *TPG_BGP     = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x20); // Background Pattern
6 volatile uint32_t *TPG_FGP     = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x28); // Foreground Pattern
7 volatile uint32_t *TPG_MS      = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x38); // Motion Speed
8 volatile uint32_t *TPG_CF      = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x40); // TPG Color Format
9 volatile uint32_t *TPG_BOX_SIZE = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x78);
10 volatile uint32_t *TPG_BOX_COLOR_Y = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x80);
11 volatile uint32_t *TPG_BOX_COLOR_U = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x88);
12 volatile uint32_t *TPG_BOX_COLOR_V = (volatile uint32_t*) (config-
>uBaseAddr_TPG_PatternGenerator + 0x90);
13
14 xil_printf("Test Pattern Generator Initialization ...\n\r");
15
16 // Direct Memory Mapped access of TPG configuration registers
17 // See TPG data sheet for configuring the TPG for other features
18 TPG_Act_H[0] = 0x438; // Active Height
19 TPG_Act_W[0] = 0x780; // Active Width
20 TPG_BGP[0]   = 0x09; // Background Pattern
21 TPG_FGP[0]   = 0x01; // Foreground Pattern
22 TPG_MS[0]    = 0x04; // Motion Speed
23 TPG_BOX_SIZE[0] = 100;
24 TPG_BOX_COLOR_Y[0] = 167;
25 TPG_BOX_COLOR_U[0] = 120;
26 TPG_BOX_COLOR_V[0] = 8;
27 TPG_CF[0]      = 0x02; // TPG Color Format
28 TPG_CR[0]      = 0x81; // TPG Control

```

A picture of the output of this change is shown below:



Software Only Change (TPG Registers not Modified)

For the software only change, we decided to read out the pixel colors in the YUV format and halve the luminance. We believed that knowing the YUV format early on in the lab would be beneficial later when we have to implement SW demosaicing. The code that reads the YUV data, halves the luminance, and then writes it back is shown below:

```
1  for (i = 0; i < 1920*1080; i += 2)
2      {
3          uint8_t u, v = 0;
4          uint16_t y = 0;
5
6          u = (pS2MM_Mem[i] & 0xFF00) >> 8;
7          v = (pS2MM_Mem[i + 1] & 0xFF00) >> 8;
8          y = (pS2MM_Mem[i] & 0xFF) | ((pS2MM_Mem[i + 1] & 0xFF) << 8);
9
10         // Half luminance
11         y /= 2;
12
13         // Set from YUV values.
14         pMM2S_Mem[i] = (u << 8) | (y & 0xFF);
15         pMM2S_Mem[i + 1] = (v << 8) | ((y & 0xFF00) >> 8);
```

The YUV 422 format is described later in this report.

A picture of the output from this change is shown below:



Though it is quite hard to see in this image, the luminance has be halved. My phone camera dod not pick this up well.

In the (.xdc) constraints file, what does the `_p` and `_n` pairing of signals signify, and what this configuration is typically used for?

In the constraints file, the `_p` and `_n` suffix pairs indicate differential signaling, specifically LVDS (Low-Voltage Differential Signaling). This is confirmed by the `IOSTANDARD` setting for these signals:

```
1 set_property IOSTANDARD LVDS_25 [get_ports IO_VITA_CAM_clk_out_*]
2 set_property IOSTANDARD LVDS_25 [get_ports IO_VITA_CAM_sync_*]
3 set_property IOSTANDARD LVDS_25 [get_ports IO_VITA_CAM_data_*]
```

What LVDS Is and How Does It Work?

LVDS uses a pair of complementary signals that are transmitted on two separate traces:

- The `_p` suffix indicates the positive/true signal
- The `_n` suffix indicates the negative/complement signal

The actual data is determined by the voltage difference between these two signals rather than the absolute voltage level. Typically, a small voltage difference (around 350mV) is used, where:

- A positive difference represents a logical '1'
- A negative difference represents a logical '0'

Why Is LVDS Used for the Camera Interface?

LVDS is used for the VITA camera interface for several important reasons:

1. **High-Speed Data Transfer:** The VITA-2000 sensor needs to transfer large amounts of image data quickly. LVDS supports high-speed data rates (often above 1 Gbps per pair) as seen in the 2.692 ns clock period (approximately 371 MHz).
2. **Noise Immunity:** Since LVDS relies on the difference between two signals rather than absolute voltage, it's highly resistant to common-mode noise that would affect both lines equally.
3. **Low EMI:** The differential pairs carry equal and opposite currents, causing the electromagnetic fields to cancel out, which reduces electromagnetic interference.
4. **Signal Integrity:** The constraint file also sets differential termination for these signals:

```
1  set_property DIFF_TERM true [get_ports IO_VITA_CAM_clk_out_*]
2  set_property DIFF_TERM true [get_ports IO_VITA_CAM_sync_*]
3  set_property DIFF_TERM true [get_ports IO_VITA_CAM_data_*]
```

This ensures proper signal integrity by eliminating reflections.

In this design, LVDS is specifically used for:

- Clock signals (`IO_VITA_CAM_clk_out_p/n`) - to provide a stable, clean timing reference
- Synchronization signals (`IO_VITA_CAM_sync_p/n`) - for frame timing synchronization
- Data lines (`IO_VITA_CAM_data_p/n`) - carrying the actual pixel data from the sensor

Why are we appending 10000000 to the output of the VITA-2000 camera? Also, why would it not make sense to

append 00000000?

The 0x10000000 offset creates a safe "sandbox" for the video frame buffers that won't interfere with other memory usage in the system. This is particularly important for video processing which involves large, continuous memory accesses through DMA that could otherwise corrupt system memory.

It would not make sense to append 0x00000000 to the output of the VITA-2000 camera because it would almost certainly result in memory corruption as the video data would overwrite critical program memory.

Why at this point, does the camera had no color?

Before implementing the demosaicing algorithm and software processing, the camera had no color because the raw data from the camera sensor is in a Bayer pattern format rather than a processed RGB or YCbCr color format.

Bayer Pattern Sensor Format:

The VITA-2000 camera sensor uses a Bayer filter pattern. This means each pixel captures only one color component (Red, Green, or Blue) arranged in a specific pattern.

Direct Raw Data Display: Without the demosaic and color processing enabled, the system is directly displaying the raw Bayer pattern data, which appears as a grayscale or monochrome image. Each pixel only contains intensity information for a single color, but the display treats it as luminance-only data.

Software Demosaicing Implementation

To implement our software demosaicing C code, we simply ran it in the main `camera_loop` function, making sure to provide pointers to the data coming in and where the demosaiced data should go to. Subgroup B's code for this is shown below:

```
1  set_park_frame(&(config->vdma_hdmi), 0, XAXIVDMA_WRITE);
2
3      // Wait until frame zero is being written to.
4      while(get_current_frame_pointer(&(config->vdma_hdmi),
XAXIVDMA_WRITE))
5      {
6
7      }
8
9      set_park_frame(&(config->vdma_hdmi), 1, XAXIVDMA_WRITE);
10
```



```

11         // Wait until frame one is being written to.
12         while(!get_current_frame_pointer(&(config->vdma_hdmi),
XAXIVDMA_WRITE))
13         {
14
15         }
16
17         // Apply CFA
18         run_demosaicing((uint16_t*)pS2MM_Mem, (uint16_t*)pMM2S_Mem);
19
20         // Swap back and front buffers
21         u8 temp = back_buffer_frame;
22
23         back_buffer_frame = front_buffer_frame;
24         front_buffer_frame = temp;
25
26         // Have the read side park on the new front buffer
27         set_park_frame(&(config->vdma_hdmi), front_buffer_frame,
XAXIVDMA_READ);
28
29         // Wait for park frame to update.
30         while(get_current_frame_pointer(&(config->vdma_hdmi), XAXIVDMA_READ)
!= front_buffer_frame)
31         {
32
33         }
34
35         // Update pMM2S_Mem to point to back buffer.
36         pMM2S_Mem = (Xuint16 *)XAxiVdma_ReadReg(config->vdma_hdmi.BaseAddr,
XAXIVDMA_MM2S_ADDR_OFFSET+XAXIVDMA_START_ADDR_OFFSET + (back_buffer_frame *
0x4));

```

The function `run_demosaicing` applies the demosaic operation to the incoming data and the rest of the code shown here is to manage the frame buffers. To allow for an easier time measuring the performance metrics and to remove tearing, we used the frame buffers smartly.

Since the demosaicing software takes a while to run, we take a single snapshot image, store it in frame buffer 0, and then let the VDMA write the incoming stream to frame buffer 1. Then, the demosaicing software can run on frame buffer 0.

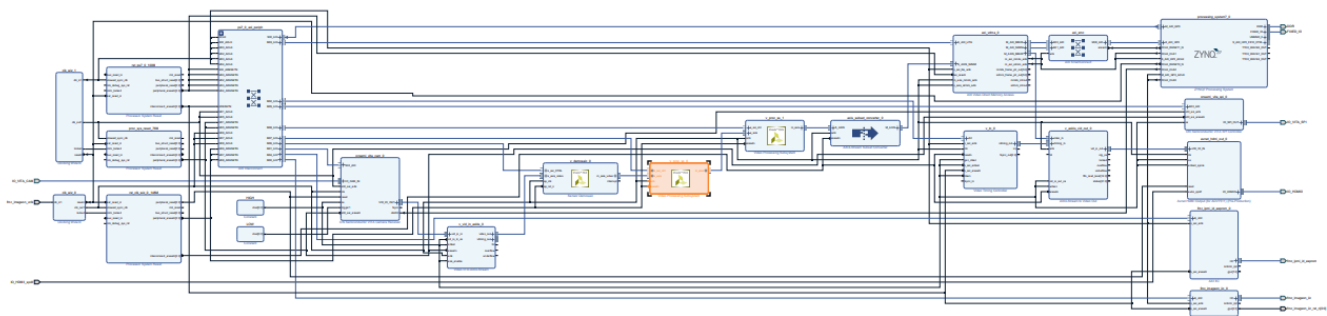
For displaying the image, we decided to treat two different read frame buffers as front and back buffers. The demosaicing code updates the back buffer while the read channel of the VDMA streams the front buffer contents to the HDMI interface. Then once the demosaicing software has ran, the buffers are swapped, which displays the latest processed frame. This worked quite

well and removed many artifacts in our images. An example of an image that resulted from the software demosaicing is shown below (image quality is a bit poor due to my phone):



Pipeline Hardware changes

Below is the resulting block diagram after adding the pipeline:



The pipeline consists of an IP block that does demosaicing to the incoming video stream and two video processing modules. One of the video processing modules converts RGB to

YCbCr 4:2:2 Format Analysis

Note from the Documentation

4:4:4 to 4:2:2 Conversion Eq from Subsystem Documentation (PG231):

$$o_{x,y} = \left[\sum_{k=0}^{N_{\text{taps}}-1} i_{x-k,y} \text{COEF}_{k,\text{HPHASE0}} \right]_0^{2^{D_w}-1}$$

Equation 3-11

This conversion is a horizontal 2:1 decimation operation, implemented using a low-pass FIR filter to suppress chroma aliasing. In order to evaluate output pixel o_{x,o_y} , the FIR filter in the core convolves $\text{COEF}_{k,\text{HPHASE0}}$, where k is the coefficient index, i_x, i_y are pixels from the input image, and $\left[\right]_m^M$ represents rounding with clipping at M , and clamping at m . DW is the Data Width or number of bits per video component. N_{taps} is the number of filter taps. The predefined filter coefficients are $[0.25 \ 0.5 \ 0.25]$.

Primary Analysis

In the YCbCr 4:2:2 format, each 32-bit word (0xF0525A52, 0x36912291, 0x6E29F029) contains data for two adjacent pixels. Breaking down these values:

```
1 // Frame #1 - Red pixels
2 for (i = 0; i < storage_size / config->uNumFrames_HdmiFrameBuffer; i += 4) {
3     *pStorageMem++ = 0xF0525A52; // Red
4 }
```

1. Red Example (0xF0525A52):

- This represents two pixels in sequence
- First 16 bits (0xF052): Y1=0xF0, Cb=0x52
- Second 16 bits (0x5A52): Y2=0x5A, Cr=0x52

```
1 // Frame #2 - Green pixels
2 for (i = 0; i < storage_size / config->uNumFrames_HdmiFrameBuffer; i += 4) {
3     *pStorageMem++ = 0x36912291; // Green
4 }
```

2. Green Example (0x36912291):

- First 16 bits (0x3691): Y1=0x36, Cb=0x91
- Second 16 bits (0x2291): Y2=0x22, Cr=0x91

```

1 // Frame #3 - Blue pixels
2 for (i = 0; i < storage_size / config->uNumFrames_HdmiFrameBuffer; i += 4) {
3     *pStorageMem++ = 0x6E29F029; // Blue
4 }

```

3. Blue Example (0x6E29F029):

- First 16 bits (0x6E29): Y1=0x6E, Cb=0x29
- Second 16 bits (0xF029): Y2=0xF0, Cr=0x29

The pattern for each color follows the YCbCr 4:2:2 format where:

- Each 16-bit word contains one Y (luminance) value and one chrominance (Cb or Cr) value
- The chrominance values alternate between Cb and Cr
- Two adjacent pixels share the same Cb and Cr values (the subsampling)

Format Structure

The 4:2:2 format packs data as follows for each pair of pixels:

```

1 [Y1][Cb][Y2][Cr]

```

Where:

- Y1: Luminance for first pixel
- Cb: Blue color difference (shared between two pixels)
- Y2: Luminance for second pixel
- Cr: Red color difference (shared between two pixels)

This format maintains full luminance resolution (the "4" in 4:2:2) while halving the horizontal resolution of the color information (the "2:2"). This works well because human vision is more sensitive to changes in brightness than in color.

For the `camera_loop()` function's conversion pass, this format would need to be maintained when processing the data, ensuring that each 32-bit word continues to represent two pixels in the YCbCr 4:2:2 format, with the appropriate luminance and chrominance values preserved during the vertical flip operation.

Hardware Pipeline Diagram

Performance

Introduction (how we measured performance)

We measured the performance of the software and hardware pipelines in this design utilizing interrupt-driven time measurements. ...

Software Pipeline

Performance

Testing Methodology

Hardware Pipeline

Performance

Testing Methodology

Bonus Credit

The following sections describe the bonus credit tasks that were completed for this project and how they were implemented/accomplished.

Various analog and digital adjustments for the gain, exposure, and other common user-configurable digital camera settings.

We implemented the following adjustments:

- Contrast
- Brightness
- Saturation

These were implemented by leveraging the second Subsystem which converts 4:4:4 to 4:2:2 and the `xvprocss.h` defined utilities.

Gain

1. Brightness Adjustment

Explanation:

The brightness level is adjusted dynamically through a dedicated function. When invoked (for example, via board buttons), it calls the hardware API to update the brightness level of the processed video image.

Code Sample:

```
1  void set_brightness(  
2      camera_config_t *config,  
3      int percent // Brightness level as a percentage  
4  )  
5  {  
6      if (config->bVerbose)  
7      {  
8          xil_printf("Setting brightness to %d\n\r", percent);  
9      }  
10     // Apply the brightness adjustment using the video processing subsystem  
11     XVprocSs_SetPictureBrightness(&proc_ss_RGB_YCrCb_444, (s32)percent);  
12 }
```

2. Contrast Adjustment

Explanation:

Similarly, contrast is adjusted using a dedicated function that calls the hardware API to modify the contrast level. This allows users to dynamically control the difference between light and dark areas in the image.

Code Sample:

```
1  void set_contrast(  
2      camera_config_t *config,  
3      int percent // Contrast level as a percentage  
4  )  
5  {  
6      if (config->bVerbose)  
7      {  
8          xil_printf("Setting contrast to %d\n\r", percent);  
9      }  
10     // Apply the contrast adjustment using the video processing subsystem  
11     XVprocSs_SetPictureContrast(&proc_ss_RGB_YCrCb_444, (s32)percent);
```

```
12 }
```

4. Saturation Adjustment

Explanation:

The saturation adjustment function controls the vividness of the colors in the video output. This function calls the corresponding hardware API to adjust the saturation level in the YCrCb color space.

Code Sample:

```
1  void set_saturation(  
2      camera_config_t *config,  
3      int percent // Saturation level as a percentage  
4  )  
5  {  
6      if (config->bVerbose)  
7      {  
8          xil_printf("Setting saturation to %d\n\r", percent);  
9      }  
10     // Apply the saturation adjustment using the video processing subsystem  
11     XVprocSs_SetPictureSaturation(&proc_ss_RGB_YCrCb_444, (s32)percent);  
12 }
```

Using the buttons on the board (code resused from mp-1), the user can adjust the gain, contrast, brightness, and saturation.

A video mode, which records and can replay up to 5 seconds of 1080p video.

Video mode was about decreasing the delay from image to image in the play mode and increasing the number of images that can be stored in the software. This was done by removing the 2-second delay in image capture and increasing the heap size to correspond with the increased number of stored images.

Code Sample:

```
1  _HEAP_SIZE = DEFINED(_HEAP_SIZE) ? _HEAP_SIZE : 0x19000000;
```

A digital zoom mode, which uses the up and down buttons to zoom in and out of the current scene.